

AI Architect, Amin Azimi — End-to-End System Development

Azimi Innovation Lab

Evaluierungsdatum: 5. September 2026

Technische Architektur- & Agentensystem-Evaluierung - IBM Bob

Erfahrungsbasierte Bewertung im Kontext der IBM SkillsBuild AI Builders Challenge und der Entwicklung einer Enterprise AI-Powered Supply Chain Optimization Platform.

GESAMTEINSCHÄTZUNG

IBM Bob erwies sich in der praktischen End-to-End-Entwicklung als leistungsfähiger, zugänglicher und produktiver Engineering-Agent. Besonders stark waren die schnelle Umsetzung umfangreicher Spezifikationen, die Fähigkeit zur strukturierten Repository-Arbeit und die niedrige Interaktionshürde. Gleichzeitig zeigten sich Grenzen, die bei langen, zustandsreichen und release-kritischen Projekten relevant werden: Kontinuität des Arbeitskontexts, Abhängigkeit von Credit-/Trial-Verfügbarkeit, die Notwendigkeit unabhängiger Endverifikation sowie Risiken bei zu breit formulierten autonomen Änderungsaufträgen.

Kontext und Bewertungsgrundlage

Ich habe IBM Bob im Rahmen einer Challenge eingesetzt, bei der die Nutzung des Agenten ein zentraler Bestandteil des Entwicklungsprozesses war. Die Plattform wurde aus vier von mir vorbereiteten Design- und Implementierungspaketen heraus end-to-end aufgebaut. Die Bewertung basiert deshalb nicht auf einer Demo oder einem kurzen Benchmark, sondern auf einem realen, mehrstufigen Softwareprojekt mit Backend, API, Datenbank, Authentifizierung, Containerisierung, Frontend, Dokumentation, Tests und finaler Integrationsarbeit.

Wichtig ist die Trennung zwischen Agentenqualität, Produkt-/Nutzungsgrenzen und Fehlern bzw. Integrationslücken im erzeugten Projektzustand. Nicht jedes im Projekt beobachtete Problem ist automatisch ein intrinsischer Defekt des Modells. Diese Evaluation kennzeichnet daher, was tatsächlich beobachtet wurde und wo nur eine technische Ursache bzw. Zuordnung plausibel ist.

Kurzurteil

Meine Erfahrung ist insgesamt positiv. Bob kann bei klaren Spezifikationen sehr viel Engineering-Arbeit beschleunigen. Für komplexe Enterprise-Projekte sollte er jedoch nicht als alleinige Freigabeinstanz behandelt werden. Seine höchste Wirksamkeit entsteht in einem kontrollierten Workflow: klarer Scope, Read-before-Write, kleine Changesets, Diff-Review, reproduzierbare Tests und eine menschliche Final Authority.



1. Stärken des Agenten im realen Projektbetrieb

Produktivität und niedrige Bedienhürde

Die Zusammenarbeit war schnell und vergleichsweise unkompliziert. Umfangreiche Anforderungen konnten in konkrete Repository-Strukturen, Module und Implementierungsschritte übersetzt werden. Gerade für die frühe und mittlere Entwicklungsphase reduzierte dies die Zeit zwischen Architekturidee und ausführbarem System erheblich.

End-to-End-Fähigkeit

Bob arbeitete nicht nur an isolierten Codefragmenten, sondern über mehrere Schichten hinweg: API, Services, Persistence, Security, Migrationen, Containerisierung, Frontend und Dokumentation. Für ein Challenge-Projekt mit begrenzter Zeit war diese Breite ein wesentlicher Vorteil.

Umsetzung strukturierter Vorgaben

Die vier ursprünglichen Design Packages boten einen klaren Rahmen. Bob konnte daraus große Teile der Plattform ableiten und umsetzen. Diese Erfahrung spricht dafür, dass die Qualität des Ergebnisses stark von präzisem Requirement Engineering und sauber begrenzten Aufträgen profitiert.

Geeignet als Engineering-Executor

Besonders überzeugend ist Bob dort, wo Ziel, Scope, Dateien und Akzeptanzkriterien bereits definiert sind. In dieser Rolle kann der Agent repetitive und umfangreiche Engineering-Arbeit beschleunigen, während Architektur- und Freigabeentscheidungen beim verantwortlichen Engineer bleiben.

01	Schnelle Umsetzung Große Spezifikationspakete konnten zügig in eine lauffähige Projektstruktur überführt werden.
02	Breite technische Abdeckung Backend, API, Datenbank, Security, Docker, Frontend, Tests und Dokumentation wurden im selben Projektkontext bearbeitet.
03	Gute Interaktion Der Workflow war zugänglich und für iterative Engineering-Aufgaben angenehm.
04	Hoher Hebel bei klaren Anforderungen Je präziser Scope und Akzeptanzkriterien, desto kontrollierbarer und wertvoller die Agentenleistung.



2. Beobachtete Grenzen, Risiken und Problemfelder

Die folgende Übersicht nennt die wichtigsten Beobachtungen nach Wirkung. Die Einstufung beschreibt das Risiko für ein langlaufendes Enterprise-Projekt - nicht eine pauschale Qualitätsnote des Agenten.

A	**Kontextkontinuität**	**HOCH**
	Bei langen Entwicklungsverläufen darf kritisches Projektwissen nicht ausschließlich im Agentenkontext verbleiben. Nach Unterbrechungen entsteht sonst Rekonstruktionsaufwand.	
B	**Credit-/Trial-Abhängigkeit**	**HOCH**
	Die verfügbare Arbeitszeit des Agenten war im Challenge-Verlauf begrenzt. Wiederholte Credit-Aktivierungen erzeugten Zeitdruck in der Schlussphase.	
C	**Finale Verifikation**	**HOCH**
	Ein 'fertig' oder 'getestet' gemeldeter Zustand ersetzt keine unabhängige End-to-End-Verifikation des tatsächlich laufenden Systems.	
D	**Breite autonome Changes**	**MITTEL-HOCH**
	Breite Prompts können zu großen Changesets führen. In komplexen Repositories steigt dadurch das Risiko unbeabsichtigter Seiteneffekte.	
E	**Environment-/Bootstrap-Klarheit**	**MITTEL**
	Der spätere lokale Lauf zeigte offene Punkte bei SECRET_KEY, Datenbankinitialisierung und Initial-Admin-Bootstrap.	
F	**Deployment-/Nginx-Integration**	**MITTEL**
	Beim realen Start traten Integrationsprobleme auf: Nginx-Bind-Mount/Config, fehlendes API-Proxying und daraus folgende HTTP-Fehler.	



3. Fallstudie: Was die reale End-to-End-Verifikation sichtbar machte

Ein besonders aussagekräftiger Teil der Evaluation war die Wiederinbetriebnahme der Plattform für die finale Demonstration. Dabei entstand eine nachvollziehbare Fehlerkette. Sie zeigt, warum Agentenberichte durch Runtime-Evidence ergänzt werden müssen.

1	Frontend gestartet HTTP 405 Nginx nahm POST /api/v1/auth/login entgegen, leitete /api jedoch nicht an das Backend weiter.
↓	
2	API-Proxy ergänzt HTTP 500 Nach korrektem Proxying erreichte der Request das Backend; dort fehlte die Datenbankschema-Initialisierung.
↓	
3	Alembic Migration HTTP 401 Nach `alembic upgrade head` existierten die Tabellen. Der Fehler wandelte sich in einen erwartbaren Auth-Fehler.
↓	
4	Users geprüft 0 Benutzer Die Auth-Pipeline funktionierte, aber ein belastbarer Initial-Admin-/Bootstrap-Pfad war im laufenden Zustand noch nicht abgeschlossen.

Engineering-Schlussfolgerung: Die Sequenz 405 → 500 → 401 war kein einzelner Defekt, sondern eine schrittweise Offenlegung mehrerer Integrationszustände. Entscheidend ist: Erst der reale Lauf über Browser, Nginx, Backend und PostgreSQL zeigte den tatsächlichen Release-Zustand.

Zuordnung mit Vorsicht: Der leere bzw. als Verzeichnis vorliegende `nginx.conf`-Mount kann durch Docker-Bind-Mount-Verhalten entstanden sein; ohne zusätzliche Provenienz-Evidence sollte dies nicht pauschal als Bob-Defekt bezeichnet werden. Bewertet wird hier deshalb primär die fehlende End-to-End-Absicherung des erzeugten Zustands.



4. Detaillierte technische Bewertung der Schwachstellen

4.1 Kontext- und Wissenskontinuität

Bei einem langen Projekt entsteht Architekturwissen nicht nur im Code, sondern auch in Entscheidungen, verworfenen Alternativen, temporären Workarounds und impliziten Annahmen. Sobald der Agent nicht mehr verfügbar ist oder ein neuer Arbeitskontext beginnt, kann dieses Wissen verloren gehen. Für Enterprise Engineering ist daher ein externer, versionierter Wissensanker erforderlich: Constitution, ADRs, Transfer Packages, Checkpoints und nachvollziehbare Git-Historie.

4.2 Nutzungsgrenzen als Projektrisiko

Die Credit-/Trial-Begrenzung war in meinem Challenge-Verlauf praktisch relevant. Zweimal musste zusätzliche Nutzungsfähigkeit hergestellt werden; trotzdem wurde die Schlussphase zeitlich eng. Das ist primär eine Produkt-/Servicegrenze und kein Beweis für einen Modellfehler. Für zeitkritische Challenges wirkt sie jedoch direkt auf Planung, Kontinuität und Risikopuffer.

4.3 Selbstberichtete Fertigstellung versus Release-Evidence

Der wichtigste technische Lernpunkt war, dass ein vom Agenten als weitgehend abgeschlossen oder getestet betrachteter Projektzustand nicht automatisch einem reproduzierbar release-fähigen System entspricht. Erst die spätere reale Integrationsprüfung legte Konfigurations-, Migrations- und Bootstrap-Fragen offen. Für Agentic Software Engineering muss daher gelten: Aussage < Test < reproduzierbarer Test < unveränderliche Evidence.

4.4 Scope-Kontrolle bei autonomen Änderungen

Agenten sind besonders wirkungsvoll, wenn sie handeln dürfen. Genau diese Stärke kann bei zu breitem Auftrag zum Risiko werden. Große Refactorings oder mehrere gleichzeitig veränderte Subsysteme erschweren Root-Cause-Analyse, Review und Rollback. Deshalb sollte Bob in kritischen Phasen mit begrenzten Aufgaben, expliziten Dateien, Akzeptanzkriterien und einem Review-Gate nach jedem Changeset geführt werden.

4.5 Environment, Secrets und Bootstrap

Beim lokalen Wiederanlauf war `SECRET_KEY` zunächst nicht wirksam gesetzt; eine temporäre Shell-Variable ermöglichte den Start. Anschließend zeigte die Datenbankprüfung, dass keine Benutzer vorhanden waren und kein unmittelbar nutzbarer Registration-Flow beobachtet wurde. Diese Punkte zeigen, dass Development Defaults, Secret Handling, Migration und First-Admin-Bootstrap als ein zusammenhängender, dokumentierter Startup Contract behandelt werden sollten.



5. Problemkatalog – kompakte Engineering Cards

P-01	State Continuity Gap
Beobachtung	Projektwissen darf nicht an eine einzelne Agentensitzung gebunden sein.
Empfehlung	Persistente ADRs, Checkpoints, Transfer Packages, maschinenlesbare Traceability.

P-02	Usage-Capacity Risk
Beobachtung	Credit-/Trial-Grenzen können einen kritischen Entwicklungsfluss abrupt unterbrechen.
Empfehlung	Kapazität vor Milestones planen; Puffer und agentenunabhängige Handoff-Dokumentation.

P-03	Verification Gap
Beobachtung	Agentenstatus 'done' ist kein Release-Beweis.
Empfehlung	Runtime E2E, Clean-start-Test, Test Evidence, unabhängiger Review.

P-04	Change-Surface Risk
Beobachtung	Breite Prompts erhöhen die Zahl gleichzeitig veränderter Variablen.
Empfehlung	One-scope/one-change, Diff Review, Rollback-Punkt, explizite Approval Gates.

P-05	Operational Bootstrap Gap
Beobachtung	Secrets, Migration und Initial Admin müssen reproduzierbar dokumentiert sein.
Empfehlung	Bootstrap Contract + sichere Secret-Injektion + idempotente Initialisierung.

P-06	Integration Drift
Beobachtung	Einzelne Komponenten können korrekt wirken, während die Gesamtstrecke scheitert.
Empfehlung	Browser → Nginx → API → DB als verpflichtende Release-Strecke testen.



6. Empfohlenes Betriebsmodell für IBM Bob

Aus der praktischen Erfahrung ergibt sich kein Argument gegen agentisches Engineering – im Gegenteil.

Die richtige Schlussfolgerung ist ein stärker kontrolliertes Betriebsmodell.

1	Inspect Zuerst Repository, Status, Konfiguration, Logs und relevante Dokumente lesen. Keine Mutation.
2	Report Problem, Evidenz, betroffene Dateien, Risiken und Alternativen explizit benennen.
3	Approve Menschliche Final Authority entscheidet über Scope und Änderung.
4	Change Kleinste reversible Änderung durchführen; keine opportunistischen Nebenrefactorings.
5	Verify Gezielte Tests plus relevante Regression; bei Integrationsarbeit reale Runtime-Strecke.
6	Evidence Diff, Testresultate, Logs und Entscheidung nachvollziehbar sichern.

Empfohlene Rollenverteilung

Project Owner / Final Authority: Ziel, Risikoakzeptanz und Freigabe. Principal Engineering Reviewer: Architektur, Diff, Teststrategie und Evidence. IBM Bob: Repo-aware Engineering Executor innerhalb des genehmigten Scopes.



7. Produktfeedback für zukünftige Versionen

Die folgenden Verbesserungen würden aus meiner Sicht die Eignung für lange, professionelle Engineering-Projekte deutlich erhöhen.

Priorität	Verbesserung	Nutzen
P0	Persistente, explizit exportierbare Project Memory / Decision Ledger	Weniger Kontextverlust bei Unterbrechungen und Agentenwechsel.
P0	Release Verification Mode mit Clean-Start-E2E-Check	Trennt Implementierungsabschluss von tatsächlich reproduzierbarer Betriebsfähigkeit.
P0	Strikte Change-Budget-/Scope-Grenzen	Begrenzt Dateien, Zeilen, Subsysteme oder erlaubte Operationen pro Auftrag.
P1	Native Evidence Pack	Automatische Sammlung von Diff, Tests, Logs, Versionen und Hashes pro Milestone.
P1	Bootstrap Readiness Check	Prüft Secrets, Migrationen, Seed/Admin, Ports und Reverse Proxy vor 'ready'.
P1	Credit-/Capacity Forecast für lange Tasks	Verhindert, dass ein mehrstufiger Workflow kurz vor Abschluss unterbrochen wird.

Diese Punkte sind als konstruktives Produktfeedback formuliert. Sie sollen die Stärken des Agenten nicht relativieren, sondern genau jene Eigenschaften adressieren, die bei einem Übergang von Challenge-/Prototype-Workflows zu langlebigen Enterprise-Projekten entscheidend werden.



8. Fünf Architekturideen zur Weiterentwicklung von IBM Bob

Aus meiner Sicht als AI Architect würden die folgenden fünf Erweiterungen Bob nicht nur komfortabler, sondern vor allem technisch belastbarer, auditierbarer und sicherer für langlaufende Enterprise-Engineering-Projekte machen. Die Vorschläge leiten sich direkt aus den im Projekt beobachteten Grenzen ab.

IDEA 01	Project Continuity Graph (PCG)
Konzept	Ein persistenter, versionierter Projektgraph, der Requirements, ADRs, Commits, Tests, offene Risiken, verworfene Alternativen und Agentenentscheidungen miteinander verknüpft. Vor jeder neuen Session rekonstruiert Bob daraus den gültigen Engineering-Zustand.
Engineering-Nutzen	Löst den State-Continuity-Gap strukturell statt nur über Chat-Historie. Entscheidungen werden nachvollziehbar, Agentenwechsel verlieren weniger Wissen und Widersprüche zwischen früheren und neuen Änderungen können automatisch erkannt werden.
Wirkprinzip	Contextverlust -> weniger Rekonstruktion -> höhere Konsistenz

IDEA 02	Autonomous Change Firewall & Blast-Radius Budget
Konzept	Vor jeder Mutation erzeugt Bob einen maschinenlesbaren Change Plan mit erlaubten Dateien, maximalem Diff-Umfang, betroffenen Subsystemen, verbotenen Operationen und Rollback-Punkt. Überschreitet die tatsächliche Änderung dieses Budget, stoppt der Agent fail-closed und fordert Approval an.
Engineering-Nutzen	Agentische Autonomie wird messbar begrenzt. Das reduziert unbeabsichtigte Seiteneffekte, macht Reviews kleiner und verbessert Root-Cause-Analyse sowie Rollback-Fähigkeit - besonders bei Security-, Database- und Deployment-Änderungen.
Wirkprinzip	Plan -> Budget -> Mutation -> automatische Scope-Prüfung

IDEA 03	Evidence-Gated Definition of Done
Konzept	Bob darf einen Task erst dann als DONE markieren, wenn die im Auftrag definierten Evidence Gates erfüllt sind: Build, gezielte Tests, relevante Regression, Runtime Smoke Test, Diff Review, Konfigurationsprüfung und - bei Integrationsänderungen - ein echter End-to-End-Pfad.
Engineering-Nutzen	Damit wird die Lücke zwischen „Code erzeugt“ und „System reproduzierbar funktionsfähig“ geschlossen. Genau die später sichtbare Kette 405 -> 500 -> 401 hätte dadurch früher als unvollständiger Release-Zustand erkannt werden können.
Wirkprinzip	Implementation + Verification + Evidence = Done



8. Fünf Architekturideen - Fortse zum

IDEA 04	Environment & Bootstrap Contract Engine
Konzept	Ein dedizierter Preflight-Mechanismus validiert vor Start oder Release alle betriebskritischen Voraussetzungen: Secrets, Environment Variables, Ports, Reverse Proxy, Migration Head, Seed/Initial Admin, externe Abhängigkeiten und Container-Mounts. Ergebnis ist ein reproduzierbarer Readiness Report.
Engineering-Nutzen	Viele Integrationsfehler entstehen nicht im Business Code, sondern an Systemgrenzen. Ein expliziter Bootstrap Contract macht diese Voraussetzungen testbar und verhindert, dass ein lokal scheinbar vollständiges Projekt erst in der finalen Demo an Konfiguration oder Initialisierung scheitert.
Wirkprinzip	Preflight -> Bootstrap -> Runtime Readiness

IDEA 05	Capacity-Aware Mission Planner & Safe Handoff
Konzept	Vor mehrstufigen Aufgaben schätzt Bob benötigte Agenten-/Credit-Kapazität, zerlegt große Missionen in atomare Checkpoints und erzeugt nach jedem Checkpoint automatisch ein Handoff Bundle mit Status, Diff, Tests, offenen Risiken und nächstem sicheren Schritt.
Engineering-Nutzen	Credit- oder Session-Enden werden von einem abrupten Projektabbruch zu einem kontrollierten Übergang. Das erhöht Planbarkeit in Challenges und verhindert, dass die kritischsten letzten Prozent eines Projekts unter maximalem Zeitdruck rekonstruiert werden müssen.
Wirkprinzip	Forecast -> Checkpoints -> Handoff -> Resume

Architektonische Gesamtwirkung

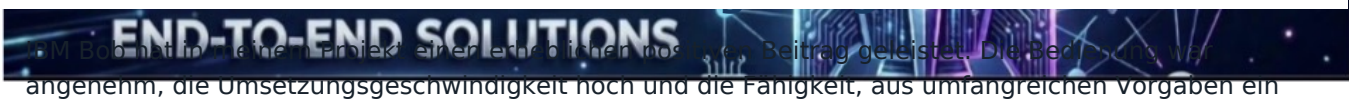
Gemeinsam bilden diese fünf Ideen eine geschlossene Kontrollschleife: Memory bewahrt den gültigen Projektzustand, die Change Firewall begrenzt Mutationen, Evidence Gates verhindern verfrühte Fertigmeldungen, der Bootstrap Contract verifiziert die reale Betriebsfähigkeit und der Mission Planner schützt die Kontinuität bei begrenzter Agentenkapazität. Dadurch würde Bob aus meiner Sicht einen deutlichen Schritt von einem leistungsfähigen Coding/Engineering-Agenten hin zu einem stärker governance-fähigen Enterprise Engineering System machen.

Empfehlung aus AI-Architecture-Sicht

Ich würde diese Funktionen nicht als optionale Komfortmerkmale, sondern als Engineering Control Plane behandeln. Für professionelle Agentic Development Systems sind Nachvollziehbarkeit, begrenzte Autonomie, reproduzierbare Verifikation und sichere Übergaben ebenso wichtig wie die reine Codegenerierungsqualität.



9. Gesamturteil und Schlussfolgerung



IBM Bob hat in diesem Projekt einen erheblichen positiven Beitrag geleistet. Die Bedienung war angenehm, die Umsetzungsgeschwindigkeit hoch und die Fähigkeit, aus umfangreichen Vorgaben ein vielschichtiges System aufzubauen, war für die Challenge außerordentlich wertvoll. Ohne diese agentische Unterstützung wäre derselbe Umfang innerhalb des verfügbaren Zeitfensters deutlich schwerer zu realisieren gewesen.

Gleichzeitig hat die praktische Arbeit gezeigt, dass ein Engineering-Agent nicht mit einer unabhängigen Release Authority gleichgesetzt werden sollte. Die letzten Prozent eines komplexen Projekts - reproduzierbarer Start, Environment, Datenbankzustand, Authentifizierungs-Bootstrap, Reverse Proxy, reale Browser-zu-Datenbank-Strecke und finale Evidence - benötigen besonders strikte Verifikation.

Die wichtigste Empfehlung lautet deshalb nicht 'weniger Agent', sondern besser kontrollierter Agent: klare Spezifikation, persistente Projektentscheidungen, begrenzte Änderungsflächen, menschliche Freigabe und reproduzierbare End-to-End-Evidence.

Bewertungsdimension	Einschätzung
Engineering-Produktivität	Sehr stark
Bedienbarkeit / Zusammenarbeit	Sehr gut
Umsetzung klarer Spezifikationen	Sehr stark
Autonomie bei engem Scope	Stark
Langzeit-Kontextkontinuität	Verbesserungspotenzial
Release-Selbstverifikation	Verbesserungspotenzial
Eignung für Enterprise-Projekte	Hoch - mit Governance und Human Review

Amin Azimi
AI Architect - End-to-End System Development
Azimi Innovation Lab

Diese Evaluation dokumentiert meine praktische Projekterfahrung und ist als konstruktive technische Rückmeldung zu verstehen. Sie unterscheidet bewusst zwischen beobachteten Projektzuständen, Produkt-/Nutzungsgrenzen und eindeutig zurechenbaren Agentenfehlern.